Anatomy of a Regional

Five hours, three brains, one keyboard

Learning goals

- Know the rules that shape strategy: scoring, penalty, freeze, notebook
- Recognise the typical *shape* of a regional problem set
- Run a fast, disciplined triage in the first 20 minutes
- Operate a 3-person / 1-computer team without wasting machine time
- Decide when to switch, when to pair-debug, and when to give up

Where you are now

- Year Curriculum done: you *know* BFS, segment trees, DP, flows basics
- Regionals rarely ask for a new algorithm — they ask you to
  - **recognise** a known technique under a disguise,
  - **implement** it correctly the first time,
  - **coordinate** three people around one machine
- This track: patterns at regional difficulty + the team skills that convert knowledge into balloons

The rules that matter

Rule	Consequence
Rank = problems solved, then penalty	One more solve beats any amount of speed
Penalty = Σ (accept minute + 20 × rejected tries)	Wrong answers on *solved* problems cost 20 min each
Unsolved problems cost nothing	Wrong tries on a problem you never solve are free
Scoreboard frozen in the last hour	You stop seeing others' late progress
One computer per team	Machine time is the bottleneck resource
Printed notebook (team reference document)	Bring tested code, not memory

Penalty arithmetic

Team	Solves (minute, wrong tries)	Solved	Penalty
X	A(12,0) B(45,2) C(170,0)	3	12 + 85 + 170 = **267**
Y	A(20,0) B(60,0) C(110,1)	3	20 + 60 + 130 = **210**

- Y ranks above X despite X's faster first solve
- Lesson: **early easy problems + no wrong tries** dominate the tiebreak
- Two WAs ≈ 40 minutes ≈ solving a problem much later

Shape of a typical regional set

```
10–13 problems, unsorted (often alphabetical)

~2–3  "everyone solves"      ad hoc, simulation, simple math
~3–4  "medium"               greedy w/ proof, BFS/Dijkstra model, DP
~2–3  "medium-hard"          DP + data structure, geometry, NT
~2–3  "hard"                 decides 1st place; few teams solve
```

- Gold-medal teams solve the first three tiers
- Most teams' fate is decided in tier 2: the **medium** problems

Problem archetypes you will meet

- Implementation / simulation (Lecture 2)
- Greedy with an exchange argument (Lecture 3)
- Graph *modeling*: the graph is not given (Lecture 4)
- DP with a twist: optimisation, counting, reconstruction (Lecture 5)
- Sweep line + segment tree (Lecture 6)
- Number theory: CRT, φ, inclusion–exclusion (Lecture 7)
- Geometry with integer coordinates (Lecture 8)
- Strings: hashing, palindromes (Lecture 9)

Minute 0–20: triage

1. Split the stack: P1 reads A–D, P2 reads E–H, P3 reads I–L
2. Meanwhile one person types the template and `.bashrc` aliases
3. Each reader labels every problem:

Label	Meaning
E	Easy: I can code it now in < 15 min
M	Known technique, needs thought / care
H	No idea yet, or huge implementation

4. Fastest **E** goes to the keyboard immediately

The problem board

Keep a sheet (or whiteboard) everyone updates:

```
Prob | Label | Owner | Status         | Idea
-----+-------+-------+----------------+-------------------------
 A   |  E    | Ana   | AC 0:09        | sort + scan
 B   |  M    | Ben   | coding         | 0-1 BFS on (cell,dir)
 C   |  H    |  -    | read           | geometry? ask later
 D   |  M    | Cy    | thinking       | DP over (i, last), n=5000
 E   |  E    | Ana   | WA 0:31 print  | off-by-one? check n=1
```

- Visible state prevents two people solving the same problem
- "Idea" column lets anyone take over

Read the scoreboard

- After ~30 min, the scoreboard *is* a difficulty oracle
- Many teams solved X early → X is easy; re-read it if you labelled it H
- Nobody solved Y after 2 hours → probably hard *or* a trap
- Before the freeze: decide your final 1–2 targets
- Don't chase the leaders' hardest problem unless you have an idea

3 people, 1 computer

```
Keyboard  ── codes a problem that is fully designed on paper
Thinker 1 ── designs next problem; writes code skeleton on paper
Thinker 2 ── debugs a printed WA, or reads/verifies an idea
```

- **Rule 1:** nobody sits at the keyboard to *think*
- **Rule 2:** on WA, print the code and leave the machine
- **Rule 3:** hand-offs announced aloud ("I need 10 min for D")

Pair work patterns

- **Explain to verify:** owner explains the solution in 2 minutes; listener attacks it with small cases
- **Pair debugging on paper:** one reads code line by line, other checks against the idea
- **Second implementer:** for tricky problems, a teammate writes a brute force on paper, typed later for a stress test
- Avoid "three people on one problem" except for the last hour

Before submitting: 60-second checklist

- Samples pass (all of them, exact output format)
- Overflow: is any product / sum > $2 \cdot 10^9$? → `long long`
- Extremes: n = 1, n = max, all equal, empty answer
- Complexity check with the *max* constraints, not the sample
- Output: trailing spaces fine? precision? `YES` vs `Yes`?
- Did you remove debug output?

After a wrong answer

```
WA ─┬─ print code, leave keyboard
    ├─ re-read statement (50% of WAs are misreads)
    ├─ check limits / overflow / output format
    ├─ hand-craft tricky cases
    └─ still stuck after ~20 min? → write brute force,
       stress test when machine is free
```

- TLE: recompute complexity; check I/O speed and `endl`
- RTE: array bounds, recursion depth, division by zero

Stress testing at a regional

```
# gen.py prints a random small test; brute.cpp is obviously correct
for i in $(seq 1 500); do
  python3 gen.py $i > in
  ./sol < in > o1; ./brute < in > o2
  cmp -s o1 o2 || { echo "diff on test $i"; cat in; break; }
done
```

- Costs ~10 minutes of machine time; pays off on a WA you cannot find
- Keep the loop in your notebook so nobody types it from memory

Time plan (5 hours)

Time	Focus
0:00–0:20	Read all, label, solve the first easy
0:20–1:30	All easies, start mediums in parallel
1:30–3:30	Mediums; one person scouts a hard problem
3:30–4:00	Pre-freeze review: what can still be solved?
4:00–5:00	Commit to 1–2 problems; pair on them

- Re-plan every hour: a 2-minute stand-up around the board

When to switch problems

- You have been stuck > 30–40 min with no new idea → swap with a teammate
- A WA you cannot explain after 20 min → stress test or hand over
- The machine is idle and someone has a designed solution → they take it
- Last hour: switch only toward a problem with a *concrete* plan

Team roles that work

Role	Typical strengths
Implementer	fast, accurate typing; simulation, data structures
Mathematician	number theory, combinatorics, geometry proofs
Generalist / captain	reads the scoreboard, tracks the board, graphs/DP

- Roles are defaults, not walls
- Everyone must be able to code everything in the notebook

The team notebook (TRD)

- Only **tested** code; each snippet compiled and verified on a problem
- Suggested contents: template, DSU, segment tree (+lazy), Dijkstra, Dinic, LCA, KMP/Z/hashing, Manacher, geometry core, CRT, Miller–Rabin, FFT
- Organise by topic with a one-page index
- Note the complexity and input format for each snippet

Worked triage example

```
A  grid robot with commands              → E (simulation)
B  min cost with 1 discount coupon        → M (Dijkstra on states)
C  count strings avoiding a pattern       → M (KMP automaton DP)
D  polygon area with integer coords       → E (shoelace)
E  schedule jobs, max profit              → M (exchange + heap)
F  3D geometry, many cases                → H (park it)
G  sum of gcd(i, n) for n ≤ 1e12          → M (φ over divisors)
```

- Order: A, D first → B, E, G in parallel thinking → C → F last

Code: scoreboard ranking

```
sort(order.begin(), order.end(), [&](int a, int b) {
    if (solved[a] != solved[b]) return solved[a] > solved[b];
    if (penalty[a] != penalty[b]) return penalty[a] < penalty[b];
    return a < b;
});
// on AC: penalty += minute + 20 * wrongTriesOnThisProblem
// submissions after AC are ignored; CE usually not penalised
```

- Full program in the notes — a nice warm-up *and* a model of the rules

Common team failures

- Everyone reads one problem in parallel; nobody reads the rest
- Coding at the keyboard while designing → idle-thinking machine time
- Resubmitting the same code "maybe it was the judge"
- Ignoring an easy problem because nobody re-read it after labelling H
- Last hour panic: three half-written solutions, zero accepted

Practice plan for this track

- Weekly: one full 5-hour virtual regional as a team (one computer!)
- Within 48 hours: **upsolve** every problem solved by ≥ 30% of teams
- Rotate the keyboard role between contests
- Archives: NWERC, SWERC, NAC / NA regionals, CERC, Asia regionals (Codeforces Gym, Kattis, `icpc.global`)

Practice problems (warm-up set)

- Kattis `different` — A Different Problem (overflow check)
- Kattis `pot`, `carrots`, `quadrant` — speed-typing warm-ups
- Codeforces 4C — Registration System
- CSES 1619 — Restaurant Customers
- Codeforces 1B — Spreadsheets (classic implementation trap)
- A full NWERC set on Codeforces Gym as a team, under contest rules

Summary

- Score = solves first, then penalty; wrong tries on solved problems hurt
- Triage early, keep a visible problem board, read the scoreboard
- Keyboard is for typing designed solutions only
- On WA: print, leave, re-read, stress test
- Next lecture: implementation-heavy problems — the easiest place to lose time